

Module 14: Locality of Reference and Cache Organization

1. Principle of Locality

- **Temporal Locality:** Same location accessed repeatedly soon (Loops, counters).
- **Spatial Locality:** Nearby consecutive memory locations accessed soon (Arrays, sequential instructions).

2. Cache Mapping Techniques

Scheme	Mapping Rule	Pros & Cons
Direct	Line = Block mod N	Fastest & cheapest; high conflict misses
Fully Associative	Any block in any line	No conflict misses; expensive hardware comparators
Set-Associative	Set = Block mod NumSets	Best industry compromise between cost & performance

Locality of Reference & Cache Memory Organization

Principle of Locality

1. Temporal Locality (Locality in Time)

- Concept: Jo memory location abhi access hui hai, woh near future me DOBARA access hone ki **high probability rakhti hai.**
- Examples: Loops (for, while), loop index counters, subroutine/function code, top of stack.

2. Spatial Locality (Locality in Space)

- Concept: Jo location abhi access hui hai, uske AAS-PAAS (neighboring) ke memory locations **soon access hone ki high probability rakhte hain.**
- Examples: Array sequential traversal, continuous code instruction stream, consecutive struct fields.

Cache Mapping Architectures

1. Direct Mapping

Block i maps to Cache Line = (i mod N). Fast & simple hardware.
Conflict Misses: High when multiple blocks map to same line.

2. Fully Associative Mapping

Any block can go into ANY cache line. Zero conflict misses!
Costly: Requires parallel hardware comparators for every line.

3. k-Way Set-Associative (Industry Standard)

Cache is divided into Sets; each Set contains k lines.
Block maps to Set = (Block mod NumberOfSets); inside set any line.
AMAT = Hit_Time + (Miss_Rate × Miss_Penalty)

3. AMAT Formula

AMAT = Hit_Time + (Miss_Rate × Miss_Penalty)